



U.S. General Services
Administration



Chapter 5.13: Hybrid Decision Systems Fundamentals

US General Service Administration
AI Community of Practice - March 2026

Sequential Architecture -- The Pipeline Pattern

- Components arranged in fixed processing order
- Each stage transforms input for the next stage
- Most common hybrid pattern in production systems
- Modularity allows independent component updates
- Weakness: no feedback from later stages to earlier ones

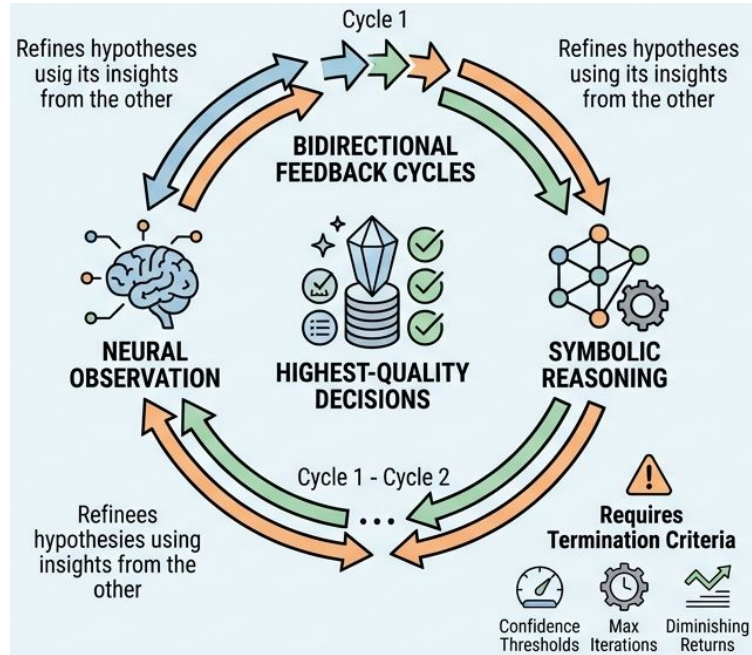


Parallel Architecture

- Multiple components analyze the same input simultaneously
- A fusion mechanism combines independent conclusions
- Redundancy improves robustness against single-component failure
- Captures complementary aspects of complex problems
- Conflict resolution needed when components disagree

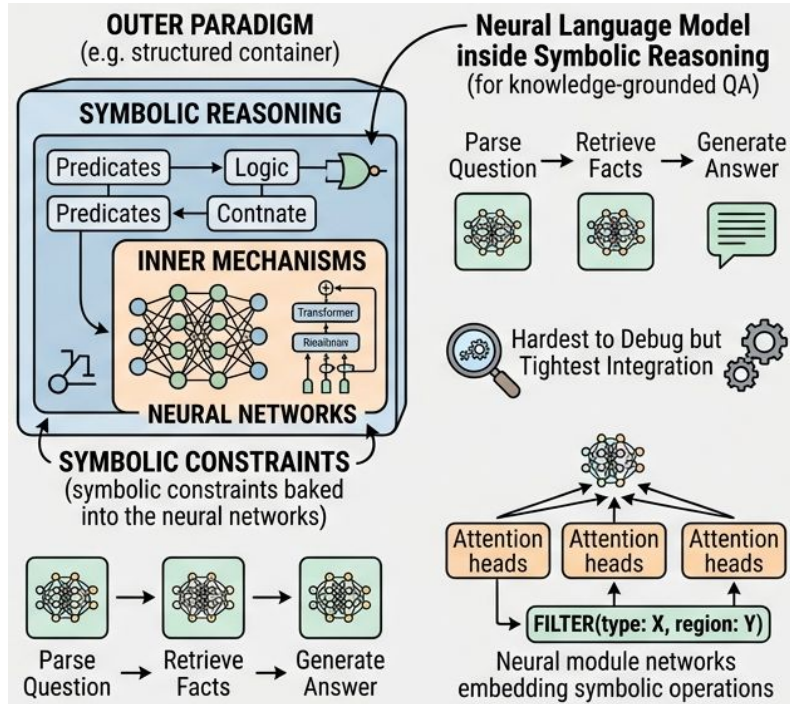


Cooperative Architecture -- Iterative Refinement



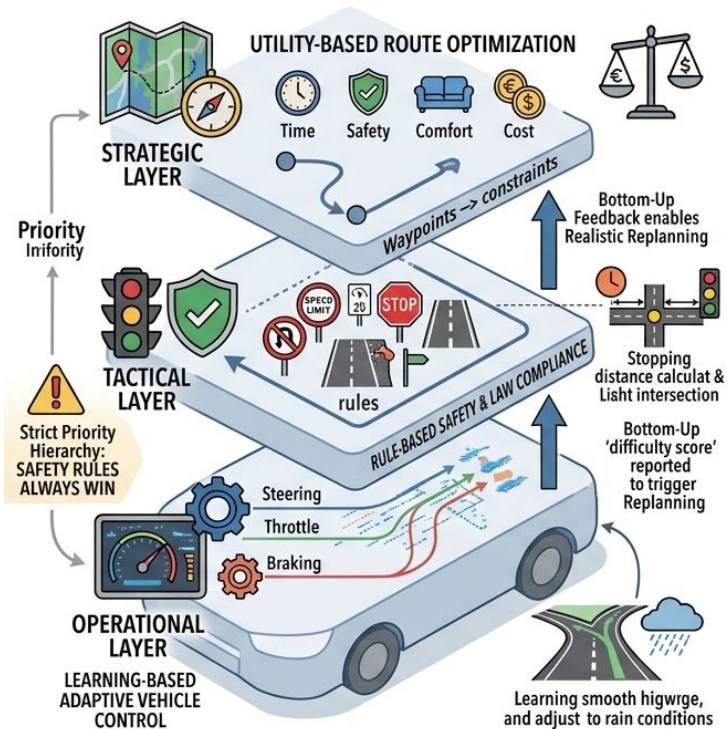
- Components exchange information across multiple cycles
- Bidirectional feedback distinguishes this from pipeline
- Each cycle refines hypotheses using other components' insights
- Produces highest-quality decisions for complex problems
- Convergence not guaranteed; requires termination criteria

Embedded Architecture -- Deep Integration



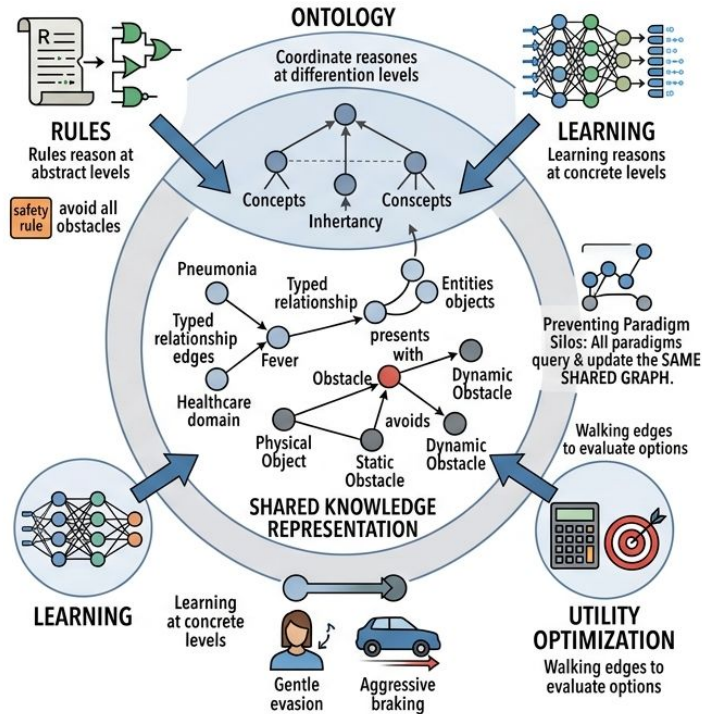
- One paradigm's mechanisms placed inside another paradigm
- Neural networks with symbolic constraints baked in
- Enables capabilities impossible with loosely coupled components
- Example: neural language models inside symbolic reasoning
- Hardest to debug but tightest integration possible

Case: The Autonomous Vehicle



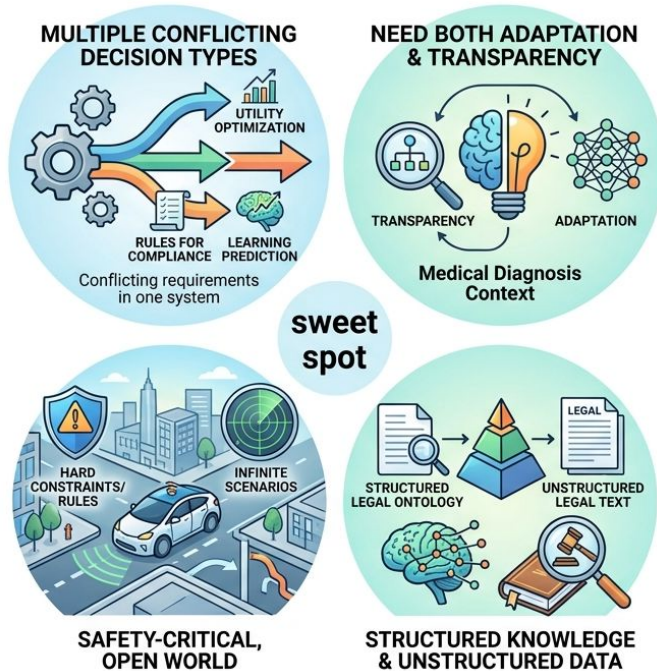
- Strategic layer: utility-based route optimization
- Tactical layer: rule-based safety and law compliance
- Operational layer: learning-based adaptive vehicle control
- Strict priority hierarchy: safety rules always win
- Bottom-up feedback enables realistic replanning

Knowledge Graphs and Ontologies as Bridges



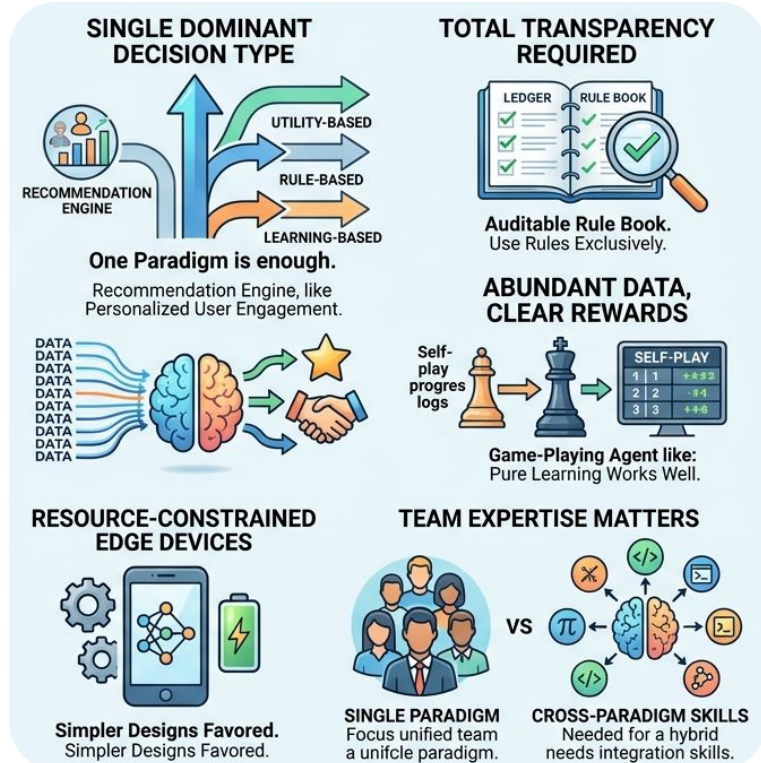
- Knowledge graphs link entities via typed relationships
- All paradigms query and update the same shared graph
- Ontologies add hierarchical concept organization
- Rules reason at abstract levels; learning at concrete levels
- Shared representation prevents paradigm silos

When Hybrid Beats Pure -- The Decision Matrix



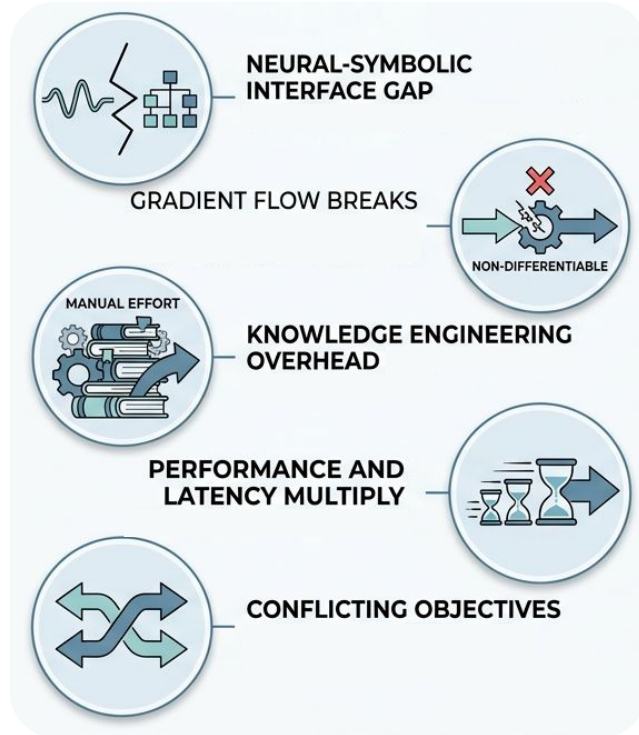
- Multiple distinct decision types with conflicting requirements
- Need both adaptation and transparency in one system
- Safety-critical systems in open-world environments
- Structured knowledge combined with unstructured data
- Four characteristics signal the "sweet spot" for hybrids

When Pure Paradigms Are Enough



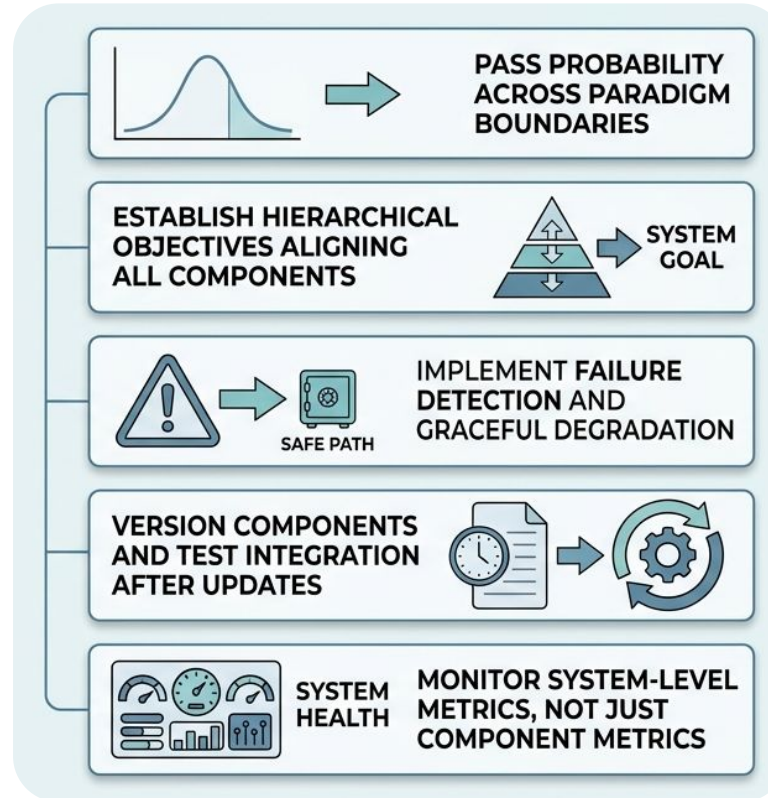
- Single dominant decision type favors one paradigm
- Total transparency required: use rules exclusively
- Abundant data, clear rewards: pure learning works well
- Resource-constrained edge devices favor simpler designs
- Team expertise matters: hybrid needs cross-paradigm skills

Five Integration Challenges That Will Bite You



- Neural-symbolic interface gap: continuous vs. discrete
- Gradient flow breaks at non-differentiable symbolic operations
- Knowledge engineering overhead for symbolic components
- Performance and latency multiply across components
- Conflicting objectives between neural and symbolic parts

Preserving Information and Maintaining Coherence



Hybrid Systems

- Hybrid systems combine paradigms for decisions beyond one approach
 - Architecture choice depends on problem decomposability and constraints
 - Shared knowledge representations prevent paradigm silos
 - Know when hybrid complexity is justified versus overkill
 - Next up: RAG, chunking, and embeddings in Chapter 6.1
- 